

Refactoring Theory Exam Booklet

Based on W5C2 Code Refactoring Slides (Martin Fowler)

1. What is Refactoring? (2 Marks)

Definition: Refactoring is a software transformation that preserves the external behaviour of software while improving its internal structure.

Purpose: Make code easier to understand, modify and maintain without changing its output.

2. Why Should We Refactor? (4 Marks)

- Improve software design
- Counter code decay (software ageing)
- Increase software comprehensibility
- Find bugs and write more robust code
- Reduce long-term maintenance cost and improve productivity

3. When Should You Refactor? (4 Marks)

- While adding new features
- During bug fixing
- During code reviews
- Apply the **Rule of Three**: first time write it, second time duplicate it, third time refactor it.

4. When Should You NOT Refactor? (2 Marks)

- When rewriting from scratch is easier than refactoring.
- When you are too close to a project deadline.

5. Categories of Refactoring (4 Marks)

Small Refactorings: Decomposing Methods, Moving Features Between Objects, Organising Data, Simplifying Conditional Expressions, Dealing with Generalisation, Simplifying Method Calls.

Big Refactorings: Tease Apart Inheritance, Extract Hierarchy, Convert Procedural Design to Objects, Separate Domain from Presentation.

6. Extract Method (2 Marks)

What? Move a code fragment into a separate method with a meaningful name.

Why? Improves clarity and removes redundancy.

7. Inline Method (2 Marks)

What? Remove a method whose body is as clear as its name and place the body directly in the caller.

Why? Removes unnecessary delegation and indirection.

8. Encapsulate Field (2 Marks)

What? Make a public field private and provide getter/setter methods.

Why? Increases encapsulation, modularity and maintainability.

9. Hide Delegate vs Remove Middle Man (Very Important)

Hide Delegate	Remove Middle Man
Keep the middle class	Remove the middle class
Client talks only to the server	Client calls the delegate directly
Increases encapsulation	Removes unnecessary indirection
Use when delegate should be hidden	Use when server only forwards calls

Exam Answer: Keep the middle class using **Hide Delegate** to increase encapsulation. Remove it using **Remove Middle Man** when it performs only simple delegation and creates unnecessary indirection.

10. Move Method (2 Marks)

Move a method to another class when it uses more features of that class than its own.

11. Extract Class (2 Marks)

Create a new class when one class is doing the work of two classes. This improves readability and reduces class complexity.

12. Inline Class (2 Marks)

If a class does very little, move all its features into another class and delete the unnecessary class.

13. Simplifying Conditional Expressions (2 Marks)

Important refactorings: Decompose Conditional, Consolidate Conditional Expression, Consolidate Duplicate Conditional Fragments, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism.

14. Push Down Method (2 Marks)

Move a superclass method to subclasses when only some subclasses require that behaviour.

15. Pull Up Method (2 Marks)

Move identical methods from subclasses into the superclass to eliminate duplication.

Quick Revision

Most Important Topics: Refactoring Definition, Why Refactor, Extract Method, Encapsulate Field, Hide Delegate vs Remove Middle Man, Extract Class, Move Method, Rule of Three.